SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS: **Total Marks: 50**

Code of Conduct:

I K. Lahari(192325118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Lahari

1. Explain Responsive Web Design. Discuss its concept, importance, CSS techniques, Flexbox and Grid, give a simple example and describe how it improves user experience.

Concept: Responsive Web Design (RWD) is an approach in which a web page automatically adjusts its layout, images and text to the screen size of the device (mobile, tablet or desktop) using a single HTML code base.

Importance: One website works on all devices, so no separate mobile site is needed. It reaches more users, reduces maintenance cost and improves search engine ranking because search engines prefer mobile-friendly sites.

CSS techniques used:

- Viewport meta tag to set the page width equal to the device width.
- Relative units such as %, em, rem and vw instead of fixed pixels.
- Media queries to apply different styles for different screen widths.
- Flexible images using max-width: 100%.

Flexbox and Grid: Flexbox is a one-dimensional layout (row or column). With flex-wrap and flex: 1 1 200px, items share the space and wrap to the next line on small screens. CSS Grid is a two-dimensional layout (rows and columns). The repeat() function and the fr unit create flexible columns, and a media query can reduce the number of columns on mobile.

Example:

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
body { font-family: "Segoe UI", Arial; margin: 0; background: #f3f6fb; }
header { background: linear-gradient(90deg, #4f46e5, #06b6d4);
  color: white; padding: 12px 20px; }
.cards { display: flex; flex-wrap: wrap; gap: 12px; padding: 15px; }
.card { flex: 1 1 200px; background: white; padding: 18px;
  border-top: 4px solid #4f46e5; border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15); }
.grid { display: grid; gap: 12px; padding: 15px;
  grid-template-columns: repeat(4, 1fr); }
.grid div { background: #06b6d4; color: white; padding: 22px;
  border-radius: 8px; text-align: center; font-weight: bold; }
.grid div:nth-child(even) { background: #4f46e5; }
@media (max-width: 600px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
```

```
}
</style>
</head>
<body>
<header><h2>Responsive Page</h2></header>
<div class="cards">
  <div class="card">Flexbox Card 1</div>
  <div class="card">Flexbox Card 2</div>
  <div class="card">Flexbox Card 3</div>
</div>
<div class="grid">
  <div>1</div><div>2</div><div>3</div><div>4</div>
  <div>5</div><div>6</div><div>7</div><div>8</div>
</div>
</body>
</html>
```

Explanation: The three Flexbox cards share the row on a wide screen and wrap one below another on a narrow screen. The grid shows 4 columns on desktop, and the media query changes it to 2 columns when the width is 600px or less.

Output:

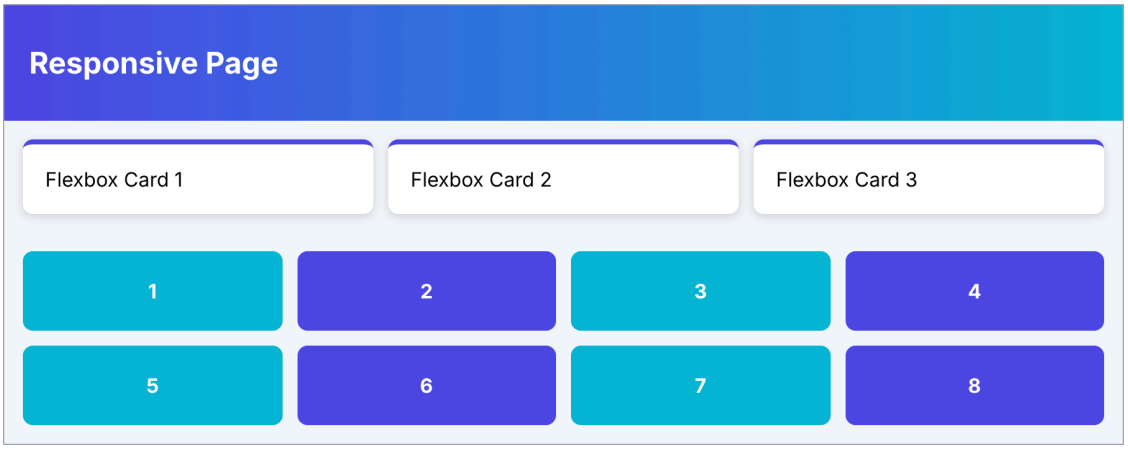


Figure 1(a): Desktop view (4 grid columns, cards in one row).

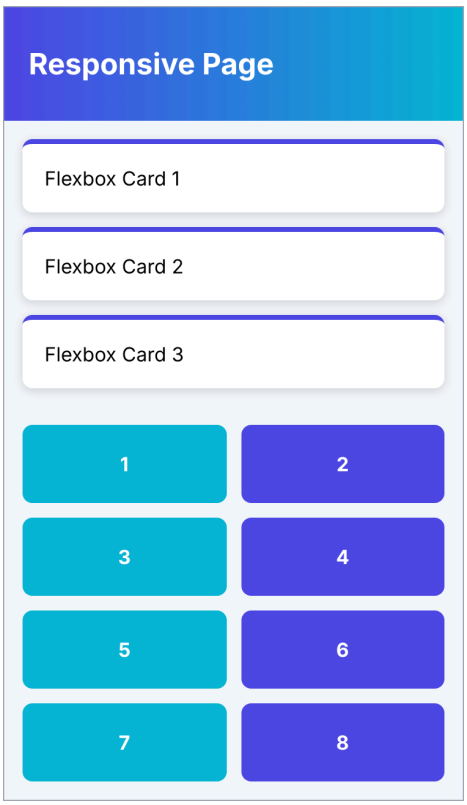


Figure 1(b): Mobile view (2 grid columns, cards stacked).

User-experience improvements: No horizontal scrolling or zooming, text and buttons are easy to read and tap, the content looks consistent on every device, and pages load and navigate more comfortably.

2. What is client-side validation? Explain its purpose, the JavaScript events used, regular expressions for email and phone number, give an example, and explain how it improves usability and security.

Purpose: Client-side validation checks the data entered by the user in the browser before it is sent to the server. It gives instant feedback, avoids unnecessary server requests and ensures that required data is in the correct format.

JavaScript events used:

- submit: checks the whole form when the user submits it.
- input / keyup: validates while the user is typing.
- blur: validates when the user leaves a field.
- change and focus: used for select boxes and for clearing old messages.

Regular expressions: The email pattern /^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i accepts text without spaces, then @, a domain name, a dot and at least two letters. The phone pattern /^[6-9][0-9]{9}$/ accepts a 10-digit mobile number starting with 6, 7, 8 or 9.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
body { font-family: "Segoe UI", Arial; background: #eef2f7;
  margin: 0; padding: 20px; }
form { background: white; width: 300px; padding: 20px;
  border-radius: 10px; box-shadow: 0 3px 10px rgba(0, 0, 0, 0.15); }
h3 { margin-top: 0; color: #4f46e5; }
input { display: block; width: 100%; box-sizing: border-box;
  margin: 5px 0; padding: 8px; border: 1px solid #aab;
  border-radius: 5px; }
button { background: #4f46e5; color: white; border: none;
  padding: 9px 18px; border-radius: 5px; margin-top: 8px; }
.err { color: #d32f2f; font-size: 13px; display: block; }
#msg { color: #2e7d32; font-weight: bold; }
</style>
</head>
<body>
<form id="f">
  <h3>Contact Form</h3>
  Email: <input id="email">
  <span class="err" id="e1"></span>
  Phone: <input id="phone">
  <span class="err" id="e2"></span>
  <button type="submit">Submit</button>
  <p id="msg"></p>
</form>
<script>
var emailRx = /^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i;
var phoneRx = /^[6-9][0-9]{9}$/;
function $(id) { return document.getElementById(id); }
function check() {
  var okE = emailRx.test($("email").value.trim());
  var okP = phoneRx.test($("phone").value.trim());
  $("e1").textContent = okE ? "" : "Enter a valid email";
  $("e2").textContent = okP ? "" : "Enter a 10-digit phone";
  return okE && okP;
}
$("f").addEventListener("submit", function (e) {
```

```
  e.preventDefault();
  $("msg").textContent = check() ? "Form is valid" : "";
});
$("email").addEventListener("input", check);
$("phone").addEventListener("input", check);
</script>
</body>
</html>
```

Explanation: The check() function tests both fields with the regular expressions and shows an error message beside any invalid field. It runs on each key press (input event) and when the form is submitted. The output below shows the result for the email "abc@" and the phone "12345".

Output:

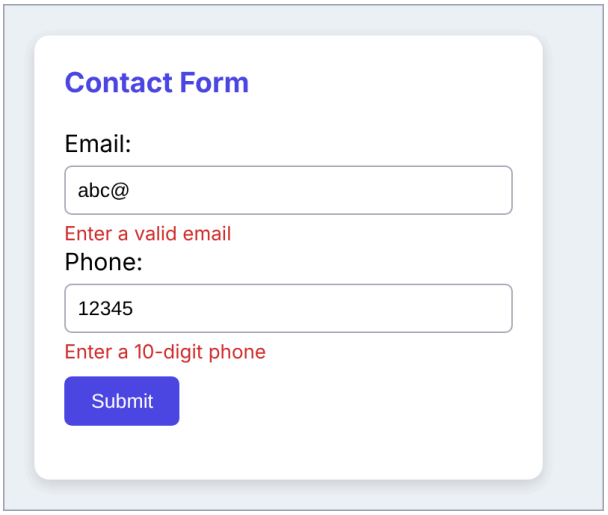


Figure 2: Error messages shown for an invalid email and phone number.

Usability: Users see mistakes immediately, correct them quickly and avoid page reloads, so forms are completed faster. Security: It blocks wrongly formatted or harmful-looking input early and reduces load on the server. However, it can be bypassed, so the server must validate the data again.

3. Explain the CSS box model and positioning. Why do layouts break, and how do media queries and best practices help to avoid this?

CSS box model: Every HTML element is a rectangular box made of four parts from inside to outside: content, padding (space around the content), border (line around the padding) and margin (space outside the border). By default (content-box) the total width = width + padding

+ border + margin. With box-sizing: border-box, the width includes the padding and border, which makes sizing easier.

CSS positioning: The position property controls where an element is placed.

- static: default, elements follow the normal flow.
- relative: moved from its normal place without affecting other elements.
- absolute: placed relative to the nearest positioned parent and removed from the normal flow.
- fixed: stays at the same place on the screen while scrolling.
- sticky: behaves as relative until a scroll point, then sticks.

Reasons why layouts break:

- Fixed widths and heights in pixels that do not fit small screens.
- Padding and border increase the width when box-sizing is not set.
- Absolute or fixed elements leave the normal flow and overlap other content.
- Uncleared floats and oversized images cause overflow.
- Missing viewport meta tag and no media queries.

Media queries: A media query applies CSS only when a condition is true, for example a screen width limit. It is used to rearrange or resize elements for different devices.

Example:

```
* { box-sizing: border-box; }
.box { width: 50%; padding: 10px;
  border: 2px solid black; margin: 10px; }
img { max-width: 100%; }
@media (max-width: 600px) {
  .box { width: 100%; }
}
```

Explanation: border-box keeps the box inside 50% width, images never exceed their container, and on screens up to 600px the box becomes full width.

Best practices: Use box-sizing: border-box for all elements, use relative units (%, rem), design mobile-first, prefer Flexbox/Grid over floats, avoid fixed heights, limit absolute positioning, and test on different screen sizes.

4. Explain the Document Object Model (DOM). Describe JavaScript DOM manipulation and dynamic updates with an example, and suggest ways to improve performance.

DOM concept: The Document Object Model is a tree-like representation of an HTML page in which every tag, attribute and text is an object (node). The browser creates it when the page loads, and JavaScript uses it to read and change the page content, structure and style.

DOM manipulation: Elements are selected using getElementById(), querySelector() and querySelectorAll(). They are changed using textContent, innerHTML, setAttribute(), style and classList. New elements are created with createElement(), added with appendChild() and removed with remove().

Dynamic updates: Using events such as click, JavaScript changes the page instantly without reloading it, for example adding items, updating counts and showing messages.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
body { font-family: "Segoe UI", Arial; background: #eef2f7;
  margin: 0; padding: 20px; }
.box { background: white; width: 320px; padding: 20px;
  border-radius: 10px; box-shadow: 0 3px 10px rgba(0, 0, 0, 0.15); }
h3 { margin-top: 0; color: #4f46e5; }
input { padding: 8px; border: 1px solid #aab; border-radius: 5px; }
button { background: #4f46e5; color: white; border: none;
  padding: 9px 16px; border-radius: 5px; }
ul { padding: 0; list-style: none; }
li { cursor: pointer; background: #e0e7ff; margin: 6px 0;
  padding: 8px 10px; border-radius: 5px; }
</style>
</head>
<body>
<div class="box">
<h3 id="title">Tasks: 0</h3>
<input id="task" placeholder="Enter a task">
<button id="add">Add</button>
<ul id="list"></ul>
</div>
<script>
```

```
var list = document.getElementById("list");
var task = document.getElementById("task");
var title = document.getElementById("title");
document.getElementById("add").onclick = function () {
  if (task.value === "") return;
  var li = document.createElement("li");
  li.textContent = task.value;
  li.onclick = function () {
    li.remove();
    title.textContent = "Tasks: " + list.children.length;
  };
  list.appendChild(li);
  title.textContent = "Tasks: " + list.children.length;
  task.value = "";
};
</script>
</body>
</html>
```

Explanation: Clicking Add creates a new li element, appends it to the list and updates the heading. Clicking a task removes it. The output shows three tasks added.

Output:

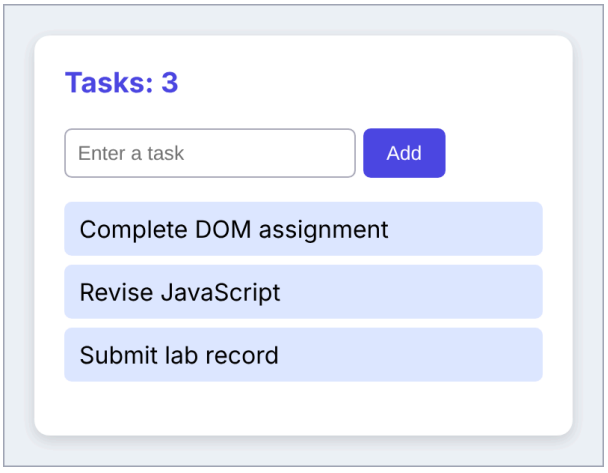


Figure 4: Tasks added dynamically and the heading updated to "Tasks: 3".

Performance improvements: Store element references in variables instead of searching repeatedly, add many elements together using DocumentFragment, use textContent instead of innerHTML for plain text, change CSS classes instead of many style properties, and use event delegation for large lists.

5. Create a responsive college website using media queries and flexible layouts.

Explanation: The page has a header with a navigation menu, a welcome section, three course boxes and a footer. Flexbox is used for the header and the course boxes, so they adjust to the available width. A media query for screens up to 700px stacks the menu links and the course boxes vertically.

Code:

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
* { box-sizing: border-box; }
body { margin: 0; font-family: "Segoe UI", Arial; background: #f3f6fb; }
header { background: #1e3a8a; color: white; padding: 10px 20px;
  display: flex; justify-content: space-between;
  align-items: center; }
nav a { color: white; margin-left: 18px; text-decoration: none; }
.hero { background: linear-gradient(135deg, #4f46e5, #06b6d4);
  color: white; text-align: center; padding: 45px 15px; }
.courses { display: flex; gap: 15px; padding: 20px; }
.course { flex: 1; background: white; padding: 20px;
  border-left: 5px solid #4f46e5; border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15); }
footer { background: #111827; color: white; text-align: center;
  padding: 12px; }
@media (max-width: 700px) {
  header { flex-direction: column; }
  nav { text-align: center; }
  nav a { display: block; margin: 8px 0; }
  .courses { flex-direction: column; }
}
</style>
</head>
<body>
<header>
  <h2>ABC College</h2>
  <nav>
    <a href="#">Home</a><a href="#">Courses</a>
    <a href="#">Admissions</a><a href="#">Contact</a>
  </nav>
```

```
</header>
<div class="hero">
  <h1>Welcome to ABC College</h1>
  <p>Quality education for a bright future</p>
</div>
<div class="courses">
  <div class="course">B.Tech Computer Science</div>
  <div class="course">B.Tech AI and ML</div>
  <div class="course">B.Tech Electronics</div>
</div>
<footer>&copy; ABC College</footer>
</body>
</html>
```

Output:

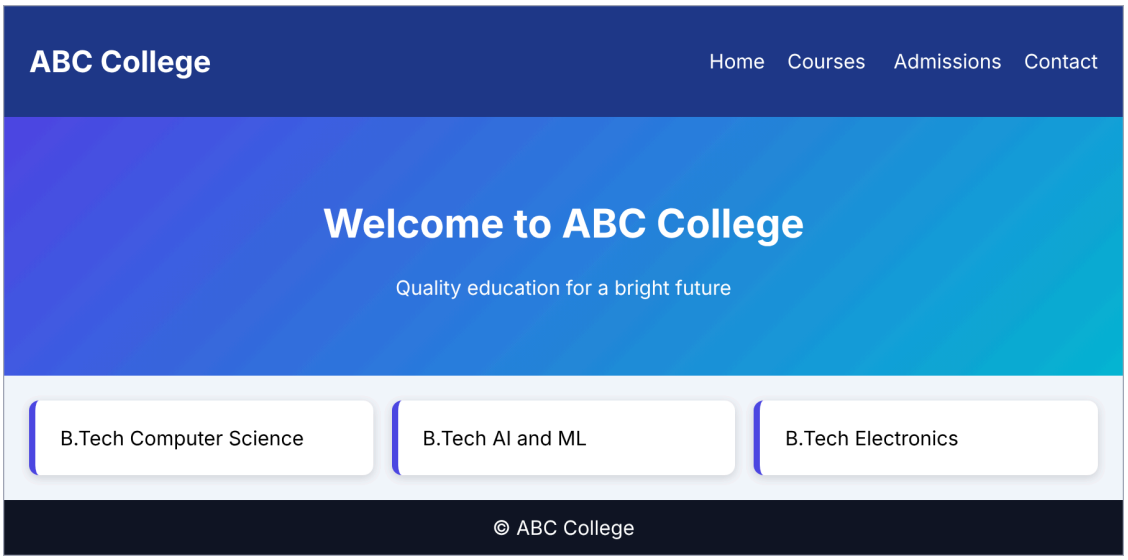


Figure 5(a): Desktop view (menu in one line, courses side by side).

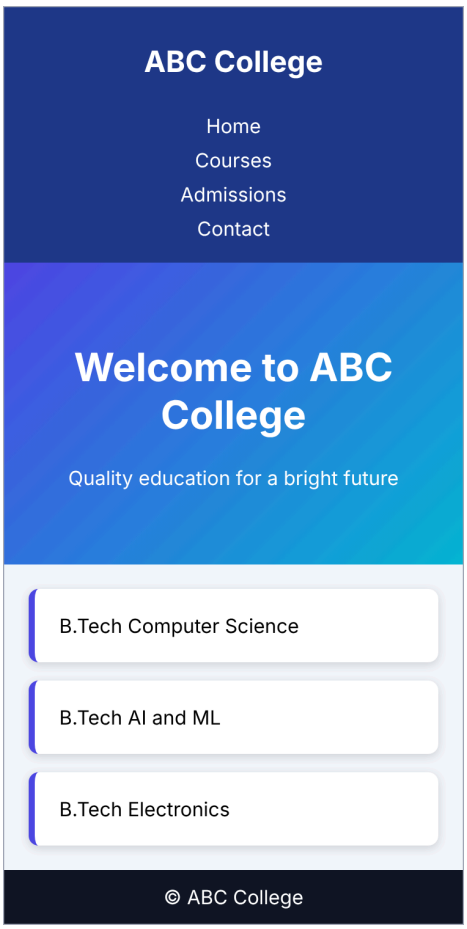


Figure 5(b): Mobile view (menu and courses stacked).

6. Improve the appearance of shopping product cards using CSS colours, borders, shadows and spacing.

Explanation: Each card has a white background on a light grey page, a thin border with rounded corners, a box-shadow for depth (stronger on hover), padding and margin for spacing, a gradient image area, a blue heading, a red bold price and a coloured button.

Code:

```
<!DOCTYPE html>
<html>
<head>
<style>
body { font-family: "Segoe UI", Arial; background: #eef2f7;
  display: flex; gap: 20px; padding: 25px; }
.card { background: white; width: 190px; padding: 15px;
  border: 1px solid #d5dbe5; border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.18); }
.card:hover { box-shadow: 0 8px 18px rgba(0, 0, 0, 0.35); }
.pic { height: 90px; border-radius: 8px; margin-bottom: 12px;
```

```
  background: linear-gradient(135deg, #4f46e5, #06b6d4); }
.card h3 { margin: 0 0 8px; color: #1e3a8a; }
.price { color: #c62828; font-weight: bold; margin: 0 0 12px; }
.card button { background: #4f46e5; color: white; border: none;
  border-radius: 6px; padding: 9px 14px; width: 100%; }
</style>
</head>
<body>
<div class="card"><div class="pic"></div>
  <h3>Headphones</h3>
  <p class="price">Rs. 1299</p>
  <button>Add to Cart</button></div>
<div class="card"><div class="pic"></div>
  <h3>Smart Watch</h3>
  <p class="price">Rs. 2499</p>
  <button>Add to Cart</button></div>
<div class="card"><div class="pic"></div>
  <h3>Backpack</h3>
  <p class="price">Rs. 899</p>
  <button>Add to Cart</button></div>
</body>
</html>
```

Output:

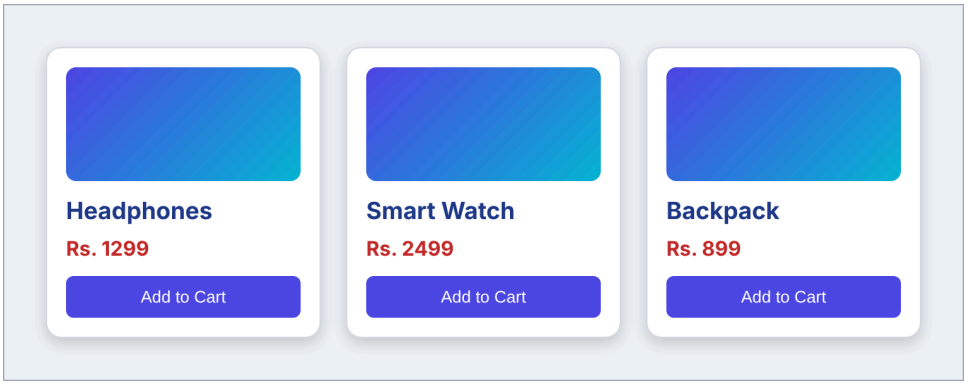


Figure 6: Improved shopping product cards.

7. A navigation menu overlaps the page content on smaller screens. Fix the problem using responsive CSS.

Problem: The menu uses position: fixed with a fixed height of 50px and floated links. On a small screen the links do not fit in one line, so the extra link goes outside the 50px menu and covers the page content. The faulty CSS is shown below.

```
body { margin: 0; font-family: "Segoe UI", Arial; }
nav { position: fixed; top: 0; width: 100%;
  height: 50px; background: #1e3a8a; }
nav a { float: left; color: white; padding: 15px;
  background: #1e3a8a; text-decoration: none; }
.content { margin-top: 50px; padding: 15px; }
```

Output before the fix:

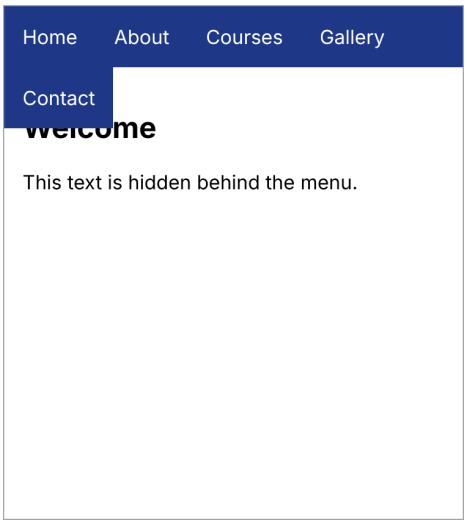


Figure 7(a): The "Contact" link overlaps the heading.

Solution: Remove the fixed position and fixed height so the menu is part of the normal flow, use Flexbox with flex-wrap, and stack the links in a column for screens up to 600px. The content then always starts below the menu.

Fixed code:

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
body { margin: 0; font-family: "Segoe UI", Arial; }
nav { background: #1e3a8a; display: flex; flex-wrap: wrap; }
nav a { color: white; padding: 15px; text-decoration: none; }
.content { padding: 15px; }
@media (max-width: 600px) {
  nav { flex-direction: column; }
  nav a { border-bottom: 1px solid #3b5bb5; }
}
</style>
</head>
```

```
<body>
<nav>
  <a href="#">Home</a><a href="#">About</a>
  <a href="#">Courses</a><a href="#">Gallery</a>
  <a href="#">Contact</a>
</nav>
<div class="content">
  <h2>Welcome</h2>
  <p>This text is now fully visible below the menu.</p>
</div>
</body>
</html>
```

Output after the fix:

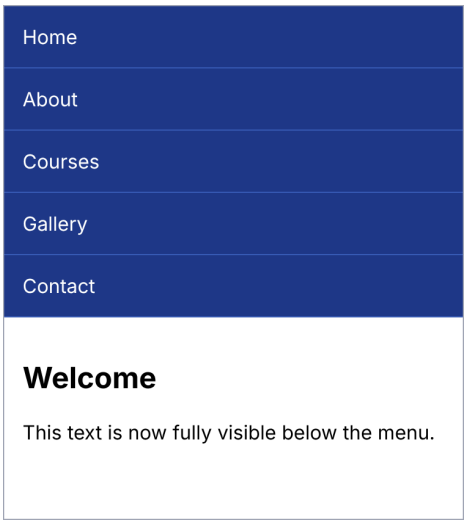


Figure 7(b): The menu is stacked and the content is fully visible.

8. Write a JavaScript program to validate empty fields in a registration form.

Explanation: The validate() function runs when the form is submitted. It checks the name, email and password fields in a loop. If a field is empty (after trim() removes spaces), an error message is shown beside it. If all fields are filled, a success message is displayed. The output shows the form submitted with all fields empty.

Code:

```
<!DOCTYPE html>
<html>
<head>
<style>
body { font-family: "Segoe UI", Arial; background: #eef2f7;
  margin: 0; padding: 20px; }
```

```
form { background: white; width: 300px; padding: 20px;
  border-radius: 10px; box-shadow: 0 3px 10px rgba(0, 0, 0, 0.15); }
h3 { margin-top: 0; color: #0d9488; }
input { display: block; width: 100%; box-sizing: border-box;
  margin: 5px 0; padding: 8px; border: 1px solid #aab;
  border-radius: 5px; }
button { background: #0d9488; color: white; border: none;
  padding: 9px 18px; border-radius: 5px; margin-top: 8px; }
.err { color: #d32f2f; font-size: 13px; display: block; }
#msg { color: #2e7d32; font-weight: bold; }
</style>
</head>
<body>
<form onsubmit="return validate()">
  <h3>Registration Form</h3>
  Name: <input id="name">
  <span class="err" id="nameErr"></span>
  Email: <input id="email">
  <span class="err" id="emailErr"></span>
  Password: <input id="pass" type="password">
  <span class="err" id="passErr"></span>
  <button type="submit">Register</button>
  <p id="msg"></p>
</form>
<script>
function validate() {
  var ids = ["name", "email", "pass"];
  var ok = true;
  for (var i = 0; i < ids.length; i++) {
    var box = document.getElementById(ids[i]);
    var err = document.getElementById(ids[i] + "Err");
    if (box.value.trim() === "") {
      err.textContent = "This field cannot be empty";
      ok = false;
    } else {
      err.textContent = "";
    }
  }
  document.getElementById("msg").textContent =
    ok ? "Registration successful" : "";
  return false;
```

```
}
</script>
</body>
</html>
```

Output:

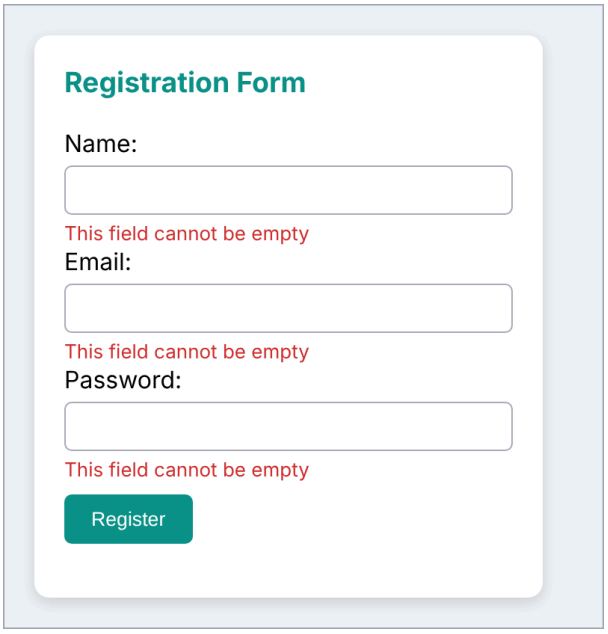


Figure 8: Error messages for empty registration fields.

9. Create an e-learning button that displays a welcome message when it is clicked.

Explanation: A button with the id "btn" is created. When it is clicked, the onclick function sets the text of the paragraph with id "msg" to the welcome message.

Code:

```
<!DOCTYPE html>
<html>
<head>
<style>
body { font-family: "Segoe UI", Arial; text-align: center;
  margin: 0; padding: 30px; background: #eef2f7; }
h2 { color: #4f46e5; }
button { background: linear-gradient(90deg, #4f46e5, #06b6d4);
  color: white; border: none; padding: 14px 30px;
  font-size: 17px; border-radius: 30px;
  box-shadow: 0 4px 10px rgba(79, 70, 229, 0.4); }
#msg { margin-top: 22px; font-size: 20px; font-weight: bold;
  color: #0d9488; }
</style>
```

```
</head>
<body>
<h2>E-Learning Portal</h2>
<button id="btn">Start Learning</button>
<p id="msg"></p>
<script>
document.getElementById("btn").onclick = function () {
  document.getElementById("msg").textContent =
    "Welcome to the E-Learning Portal!";
};
</script>
</body>
</html>
```

Output:



Figure 9: Welcome message displayed after clicking the button.

10. Update the total price of a shopping product when the quantity is changed, using JavaScript.

Explanation: The unit price is Rs. 499. Whenever the quantity changes (input event), the script reads the price and the quantity, multiplies them and displays the result in the Total. Invalid or empty quantities are treated as 1. The output shows quantity 3, so the total is 3 x 499 = 1497.

Code:

```
<!DOCTYPE html>
<html>
<head>
<style>
body { font-family: "Segoe UI", Arial; background: #eef2f7;
  margin: 0; padding: 20px; }
.box { background: white; width: 300px; padding: 20px;
```

```
  border-radius: 10px; box-shadow: 0 3px 10px rgba(0, 0, 0, 0.15); }
h3 { margin-top: 0; color: #4f46e5; }
input { width: 60px; padding: 6px; border: 1px solid #aab;
  border-radius: 5px; }
.total { background: #e0e7ff; padding: 10px; border-radius: 5px; }
</style>
</head>
<body>
<div class="box">
<h3>Wireless Headphones</h3>
<p>Price: Rs. <span id="price">499</span></p>
<p>Quantity:
  <input type="number" id="qty" value="1" min="1"></p>
<p class="total"><b>Total: Rs. <span id="total">499</span></b></p>
</div>
<script>
var qty = document.getElementById("qty");
qty.addEventListener("input", function () {
  var p = document.getElementById("price");
  var price = Number(p.textContent);
  var n = parseInt(qty.value);
  if (isNaN(n) || n < 1) n = 1;
  document.getElementById("total").textContent = price * n;
});
</script>
</body>
</html>
```

Output:

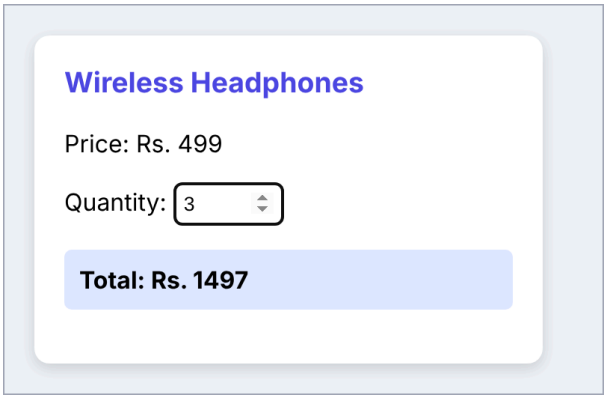


Figure 10: Total price updated to Rs. 1497 for quantity 3.